

ROB301: Introduction to Robotics, 2026F

Laboratory 1: Pose-to-pose robot control

September 1, 2026

1 Introduction

For a robot to interact with its surroundings, it must be able to move within them. The focus for this, the second laboratory exercise in the course, is pose-to-pose control for the locomotion of a robot. It is the most basic form of robotic control. In this lab, we will only be looking at feedforward or open-loop control. Naturally, we'd expect to find some shortcomings in this approach and it will be your task to investigate them. *You will be required to demonstrate your results to the TAs.*

2 Objective

The objective of this laboratory exercise is to study feedforward or open-loop pose-to-pose motion control of the TurtleBot 3 Waffle Pi. In particular, you will be required

- ▷ To study pose-to-pose control using a straight-line path
- ▷ To study pose-to-pose control using waypoints with straight-line segments
- ▷ To study pose-to-pose control using curved paths

One of the advantages of ROS is that it abstracts away the direct interfacing with the motors. Thus as studied in class you can use speed v and angular rate ω to command the robot.

You will explore different means of achieving pose-to-pose locomotion, including on-the-spot rotation, motion using waypoints, straight-line motion, and motion along curved paths. By the end of the lab, you should be comfortable with coding the TurtleBot to move to an arbitrary location and appreciate the drawbacks of motion control without feedback.

3 Equipment & Software

As introduced in Lab 0, the hardware platform used for this lab is the TurtleBot 3 Waffle Pi. In this lab, we are going to test out the accuracy of the servomotors equipped on the robot through the pose-to-pose control experiments. Since the Lab 1 tasks are similar to those in Lab 0, you can reuse the code from `motor.py` of Lab 0 in `lab2.py`. From the package 'lab_1' we can run the node 'lab_1' using ROS2.

```
ros2 run lab_1 lab_1
```

As a quick reminder from Lab 0, the following commands should be run in separate terminal windows (1) to launch the startup code on the Turtlebot, and (2) to run your custom `lab_1.py` node (you should keep these terminal windows open as they are continuously running the program):

<pre>ros2 launch turtlebot3_bringup turtlebot3_robot.launch</pre>	[TurtleBot PC]
<pre>ros2 run lab_1 lab_1</pre>	[Remote PC]

4 Assignment

For the purposes of this lab, the robot can be modeled as a “unicycle,” whose pose is given by three degrees of freedom: x , y and θ , as measured in a global reference frame. In short,

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

Recall that the model is given by

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix} \begin{Bmatrix} v \\ \omega \end{Bmatrix}$$

The column $\mathbf{x}$ will be henceforth be formally known as the robot's pose (position + orientation). The goal of this lab will be to command the robot speed and angular rate in order to move from one pose to another.

4.1 Pose-to-Pose Locomotion Using a Straight-Line Path

To demonstrate the basics of locomotion, you are required to command the robot to move from a start pose $\mathbf{x}_{\text{Start}}$ to a goal pose $\mathbf{x}_{\text{Goal}}$. The most direct way is to plan a straight-line path from the start to the goal and have the robot follow it; you may perform on-the-spot rotations as deemed necessary. Take

$$\mathbf{x}_{\text{Start}} = \begin{bmatrix} 0 \\ 0 \\ 0^\circ \end{bmatrix}, \quad \mathbf{x}_{\text{Goal}} = \begin{bmatrix} 200 \text{ cm} \\ 50 \text{ cm} \\ 135^\circ \end{bmatrix}$$

Note the error in the final pose by comparing the robot's pose with the floor markings. In addition, keep in mind that the Turtlebot's linear/angular velocities are limited: Sending command velocities greater than 0.26 m s^{-1} or 1.82 s^{-1} will cause issues with your odometry estimate since the motor speeds are limited to these values.

Before Moving On... Demonstrate this functionality to the TA.

4.2 Pose-to-Pose Locomotion Via Waypoints

Consider now a path based on a series of waypoints. In this demonstration, you are asked to move the robot through a square path, defined by waypoints A, B, C, back to its original pose. That is, let

$$\mathbf{x}_{\text{Start}} = \begin{bmatrix} 0 \\ 0 \\ 0^\circ \end{bmatrix}, \quad \mathbf{x}_A = \begin{bmatrix} 100 \text{ cm} \\ 0 \\ 90^\circ \end{bmatrix}, \quad \mathbf{x}_B = \begin{bmatrix} 100 \text{ cm} \\ 100 \text{ cm} \\ 180^\circ \end{bmatrix}, \quad \mathbf{x}_C = \begin{bmatrix} 0 \\ 100 \text{ cm} \\ 270^\circ \end{bmatrix}$$

and $\mathbf{x}_{\text{Goal}} = \mathbf{x}_{\text{Start}}$. Measure the error in pose at each of the waypoints and at the goal.

Before Moving On... Demonstrate this functionality to the TA.

4.3 Pose-to-Pose Locomotion Along a Curved Path

Unfortunately, some robots may not be capable of pivoting on the spot. (Consider nonholonomic robots!) In this case, it would be necessary to design a curved path for the robot to follow. Assuming that the minimum turning radius for the robot is 25 cm, design a path that will take it from start to goal as given in Part 4.1.

To accomplish this task, you will need to set the speed and angular rate of the robot to follow the desired path. Once again, estimate the error along the path and at the goal. Try different speed, and correspondingly different angular-rate, profiles and comment on performance.

Before Moving On... Demonstrate this functionality to the TA.

4.4 Bonus Challenge: A Complex Path

If you've finished the first four assigned tasks, try a more challenging path for the robot. Suppose we'd like the robot to follow a path described by

$$y(x) = 50 \sin\left(\frac{\pi}{25}x\right) \quad [\text{cm}]$$

Try different speed, and correspondingly different angular-rate, profiles here as well and comment on performance.

5 Closing Remarks

In this laboratory exercise, you should have been able to familiarize yourself with feedforward or open-loop control in the context of pose-to-pose control as well as following desired paths. You should also have been able to discern some of the limitations of feedforward control, upon which we will seek to improve by introducing feedback in Lab 2.